

Ex.no:2	EDA-DATA IMPORT AND EXPORT

Aim:

To inspect dataset features, detect null values, and export data into CSV, Excel, and SQLite database formats.

Algorithm:

1. Import pandas and sqlite3 libraries.
2. Read vehicle_analysis.csv using pd.read_csv().
3. Inspect data structure using .head(), .tail(), .info(), .describe(), and .isnull().sum().
4. Export DataFrame to .csv and .xlsx formats using .to_csv() and .to_excel().
5. Connect to hotel_booking.db using sqlite3 and write data into an SQL table via .to_sql().
6. Close the database connection and end execution.

Program:

```
import os

import pandas as pd

print("Dataset path:", path)

print("\nFiles in dataset:")

print(os.listdir(path))

# Find CSV file automatically

csv_file = [f for f in os.listdir(path) if f.endswith(".csv")][0]

print("\nCSV file:", csv_file)

# Load dataset

df = pd.read_csv(

    os.path.join(path, csv_file),

    low_memory=False

)

print("\nFirst 5 rows:")

display(df.head())

print("\nLast 5 rows:")
```

```
display(df.tail())
print("\nDataset information:")
df.info()
print("\nShape:")
print(df.shape)
print("\nColumns:")
print(df.columns.tolist())
print("\nData types:")
print(df.dtypes)
print("\nStatistical description:")
display(df.describe())
print("\nMissing values:")
display(df.isnull().sum())
print("\nTop 10 car names:")
display(df["Car_Name"].value_counts().head(10))
print("\nFuel types:")
display(df["Fuel_Type"].value_counts())
print("\nSeller types:")
display(df["Seller_Type"].value_counts())
print("\nTransmission types:")
display(df["Transmission"].value_counts())
print("\nUnique fuel types:")
print(df["Fuel_Type"].unique())
print("\nCorrelation:")
display(df.corr(numeric_only=True))
# Excel
df.to_excel("vehicle_analysis.xlsx", index=False)
# JSON
df.to_json("vehicle_analysis.json", orient="records", indent=4)
# CSV
df.to_csv("vehicle_analysis.csv", index=False)
print("\nCSV, Excel and JSON files created successfully.")
```

Output:

First 5 rows:

	Car_Name	Year	Selling_Price	Present_Price	Kms_Driven	Fuel_Type	Seller_Type	Transmission	Owner	
0	ritz	2014	3.35	5.59	27000	Petrol	Dealer	Manual	0	
1	sx4	2013	4.75	9.54	43000	Diesel	Dealer	Manual	0	
2	ciaz	2017	7.25	9.85	6900	Petrol	Dealer	Manual	0	
3	wagon r	2011	2.85	4.15	5200	Petrol	Dealer	Manual	0	
4	swift	2014	4.60	6.87	42450	Diesel	Dealer	Manual	0	

Last 5 rows:

	Car_Name	Year	Selling_Price	Present_Price	Kms_Driven	Fuel_Type	Seller_Type	Transmission	Owner	
296	city	2016	9.50	11.6	33988	Diesel	Dealer	Manual	0	
297	brio	2015	4.00	5.9	60000	Petrol	Dealer	Manual	0	
298	city	2009	3.35	11.0	87934	Petrol	Dealer	Manual	0	
299	city	2017	11.50	12.5	9000	Diesel	Dealer	Manual	0	
300	brio	2016	5.30	5.9	5464	Petrol	Dealer	Manual	0	

```
Dataset information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 301 entries, 0 to 300
Data columns (total 9 columns):
#   Column          Non-Null count  Dtype
---  -
0   Car_Name        301 non-null   object
1   Year            301 non-null   int64
2   Selling_Price   301 non-null   float64
3   Present_Price   301 non-null   float64
4   Kms_Driven      301 non-null   int64
5   Fuel_Type       301 non-null   object
6   Seller_Type     301 non-null   object
7   Transmission    301 non-null   object
8   Owner           301 non-null   int64
dtypes: float64(2), int64(3), object(4)
memory usage: 21.3+ KB

Shape:
(301, 9)

Columns:
['Car_Name', 'Year', 'Selling_Price', 'Present_Price', 'Kms_Driven', 'Fuel_Type', 'Seller_Type', 'Transmission', 'Owner']
```

Statistical description:

	Year	Selling_Price	Present_Price	Kms_Driven	Owner
count	301.000000	301.000000	301.000000	301.000000	301.000000
mean	2013.627907	4.661296	7.628472	36947.205980	0.043189
std	2.891554	5.082812	8.644115	38886.883882	0.247915
min	2003.000000	0.100000	0.320000	500.000000	0.000000
25%	2012.000000	0.900000	1.200000	15000.000000	0.000000
50%	2014.000000	3.600000	6.400000	32000.000000	0.000000
75%	2016.000000	6.000000	9.900000	48767.000000	0.000000
max	2018.000000	35.000000	92.600000	500000.000000	3.000000

Result: